Redis 设计与实现 精简版

Redis Design and Implementation

后端面试速查笔记

常用命令 · 数据结构 · 单机数据库 · 多机数据库

目录

常用命令速查

- 连接与服务器
- 键管理
- String
- Hash
- List
- Set
- Sorted Set
- HyperLogLog
- Bitmap
- Geo
- Stream
- Pub/Sub
- 事务与脚本

简单动态字符串（SDS）

- 结构定义
- SDS vs C 字符串
- 空间预分配
- 惰性空间释放
- 二进制安全

字典

- 数据结构
- 哈希算法
- 键冲突处理
- rehash
- 渐进式 rehash

跳跃表

- 数据结构
- 核心概念

压缩列表

- 内存布局
- 节点结构
- 连锁更新

对象系统

- 五种类型和编码
- 字符串对象的三种编码
- 有序集合为什么同时用跳表和字典

内存回收
对象共享
LRU 空转时长
数据库
数据库结构
过期键删除策略
过期键与持久化、复制
RDB 持久化
SAVE vs BGSAVE
自动保存
RDB 文件结构
AOF 持久化
工作流程
appendfsync 策略
AOF 重写
事件
文件事件
时间事件
事件调度
事务
基本流程
WATCH — 乐观锁
ACID 分析
复制
旧版复制（2.8 之前）
新版复制（2.8+，PSYNC）
命令传播阶段
Sentinel
启动与初始化
监控
下线判断
选举 Leader
故障转移
集群
节点
槽指派
MOVED 与 ASK
重新分片
故障检测

故障转移

Gossip 协议

常用命令速查

常用命令速查

面试和日常开发中最常碰到的 Redis 命令，按数据结构分类。

连接与服务器

```
AUTH password                   #
AUTH username password          # Redis 6+ ACL
PING                            #  PONG
SELECT db                       #  0~15
INFO [section]                  #
DBSIZE                          #
FLUSHDB [ASYNC]                 #
FLUSHALL [ASYNC]                #
CLIENT LIST                     #
CONFIG GET/SET param            # /
SLOWLOG GET [n]                 #
```

键管理

```
DEL key [key ...]               #
UNLINK key [key ...]            # 4.0+
EXISTS key [key ...]            #
TYPE key                        #
RENAME key newkey               #
COPY source dest [REPLACE]      # 6.2+

EXPIRE key seconds              #
PEXPIRE key ms                  #
EXPIREAT key timestamp          #
PERSIST key                     #
TTL key                         # -1 -2
PTTL key                        #

SCAN cursor [MATCH pat] [COUNT n]  #  KEYS
OBJECT ENCODING key             #
OBJECT IDLETIME key             #
```

String

最基础的类型，值可以是字符串、整数或浮点数。

```
SET key value [EX s | PX ms] [NX|XX] [KEEPTTL]  #
GET key                          #
GETDEL key                       # 6.2+
GETEX key [EX s|PX ms|PERSIST]  # 6.2+

MSET k1 v1 k2 v2 ...            #
MGET k1 k2 ...                   #

INCR key                         # +1
DECR key                         # -1
INCRBY key n                     # +n
DECRBY key n                     # -n
INCRBYFLOAT key f                # +

SETNX key value                  #  SET ... NX
SETEX key seconds value          # + SET ... EX

APPEND key value                 #
STRLEN key                       #
GETRANGE key start end           #
SETRANGE key offset value        #
```

SET 的选项组合：

- `SET key value NX EX 30`：分布式锁的标准写法——不存在才写，30 秒自动过期
- `SET key value XX KEEPTTL`：仅覆盖已有键，保留原 TTL

Hash

适合存对象，比如用户信息 `user:1001 → {name: ..., age: ...}`。

```
HSET key field value [field value ...]   # 4.0+
HGET key field                   #
HMGET key f1 f2 ...              #
HGETALL key                      # +
HDEL key field [field ...]       #
HEXISTS key field                #
HLEN key                         #
HINCRBY key field n              #  +n
HINCRBYFLOAT key field f         #  +
HKEYS key                        #
HVALS key                        #
HSCAN key cursor [MATCH] [COUNT]  #
HRANDFIELD key [count]           # 6.2+
```

List

双端队列，左右都能推入弹出。

```
LPUSH key value [value ...]     #
RPUSH key value [value ...]     #
LPOP key [count]                # count  6.2+
RPOP key [count]                #

LRANGE key start stop           # 0 -1
LINDEX key index                #
LLEN key                        #
LPOS key value                  # 6.0.6+

LSET key index value            #
LREM key count value            #  count
LTRIM key start stop            #

LMOVE src dst LEFT|RIGHT LEFT|RIGHT   # 6.2+

BLPOP key [key ...] timeout     #
BRPOP key [key ...] timeout     #
BLMOVE src dst ... timeout      #  LMOVE6.2+
```

典型用法：

- 消息队列：LPUSH 生产 + BRPOP 消费
- 最新 N 条记录：LPUSH + LTRIM

Set

无序集合，自动去重，支持交并差运算。

```
SADD key member [member ...]    #
SREM key member [member ...]    #
SISMEMBER key member            #
SMISMEMBER key m1 m2 ...        # 6.2+
SMEMBERS key                    #
SCARD key                       #
SRANDMEMBER key [count]         #
SPOP key [count]                #

SINTER key [key ...]            #
SUNION key [key ...]            #
SDIFF key [key ...]             #
SINTERCARD numkeys key [key ...] [LIMIT n]  # 7.0+
SINTERSTORE dest key [key ...]  #  dest
```

典型用法：

- 共同关注：SINTER user:A:follows user:B:follows
- 抽奖：SPOP / SRANDMEMBER

Sorted Set

有序集合，每个成员关联一个分值，自动按分值排序。

```
ZADD key [NX|XX] [GT|LT] [CH] score member [score member ...]
                                # GT/LT/6.2+
                                # CH
ZREM key member [member ...]    #
ZSCORE key member               #
ZMSCORE key m1 m2 ...           # 6.2+
ZRANK key member                #  0
ZREVRANK key member             #
ZINCRBY key increment member    #  +increment
ZCARD key                       #
ZCOUNT key min max              #  [min, max]

ZRANGE key min max [BYSCORE|BYLEX] [REV] [LIMIT offset count]
                                # 7.0
                                # ZRANGE key 0 -1
                                # ZRANGE key 0 100 BYSCORE
                                #   ZRANGE key 0 -1 REV
ZRANGESTORE dst src min max ... #  dst6.2+

ZPOPMIN key [count]             #
ZPOPMAX key [count]             #
BZPOPMIN key [key ...] timeout  #
ZRANDMEMBER key [count]         # 6.2+

ZUNIONSTORE dest numkeys key [key ...] [WEIGHTS ...] [AGGREGATE SUM|MIN|MAX]
ZINTERSTORE dest numkeys key [key ...]
```

典型用法：

- 排行榜：ZADD + ZRANGE REV + ZRANK
- 延迟队列：ZADD 用时间戳做分值 + BZPOPMIN

HyperLogLog

基数估算，不存储实际元素，标准误差 0.81%，每个键只占 12KB。

```
PFADD key element [element ...] #
PFCOUNT key [key ...]           #
PFMERGE dest key [key ...]      #  HLL
```

典型用法：统计 UV。PFADD 把每次访问的 user_id 记进去，PFCOUNT 读 UV 数。

Bitmap

字符串类型的位操作接口，每个 bit 位可以独立读写。

```
SETBIT key offset 0|1           #
GETBIT key offset               #
BITCOUNT key [start end [BYTE|BIT]]   #  1
BITPOS key 0|1 [start end [BYTE|BIT]] #  0  1
BITOP AND|OR|XOR|NOT dest key [key ...]  #
BITFIELD key GET type offset | SET type offset value | INCRBY type offset increment
                                #
```

典型用法：用户签到。每人一个 key，offset = 年内第几天。SETBIT 签到，BITCOUNT 统计次数。

Geo

基于 Sorted Set，用 Geohash 编码经纬度作为分值。

```
GEOADD key [NX|XX] [CH] lng lat member [lng lat member ...]
GEODIST key m1 m2 [m|km|mi|ft] #
GEOPOS key member [member ...]  #
GEOHASH key member [member ...] #  Geohash
GEOSEARCH key FROMMEMBER m | FROMLONLAT lng lat
    BYRADIUS r m|km | BYBOX w h m|km [ASC|DESC] [COUNT n]
                                # 6.2+ GEORADIUS
GEOSEARCHSTORE dest src ...     #  dest
```

典型用法：附近的人/店。GEOADD 存位置，GEOSEARCH BYRADIUS 查附近。

Stream

Redis 5.0 引入的日志型数据结构，比 List 做消息队列靠谱得多。支持消费者组、消息确认和待处理列表。

```
XADD key [NOMKSTREAM] [MAXLEN|MINID [=|~] threshold] *|id field value [field value ...]
                                # *  ID
XLEN key                        #
XRANGE key start end [COUNT n]  # - +
XREVRANGE key end start [COUNT n]  #
XREAD [COUNT n] [BLOCK ms] STREAMS key [key ...] id [id ...]
                                # BLOCK 0 =

#
XGROUP CREATE key group id|$ [MKSTREAM]    #
XREADGROUP GROUP group consumer [COUNT n] [BLOCK ms] STREAMS key &gt;
                                # &gt; =
XACK key group id [id ...]      #
XPENDING key group [IDLE ms] start end count [consumer]
                                #
XCLAIM key group consumer min-idle-time id [id ...]
                                #
XAUTOCLAIM key group consumer min-idle-time start [COUNT n]
                                # 6.2+
XTRIM key MAXLEN|MINID [=|~] threshold  #
XINFO STREAM|GROUPS|CONSUMERS key [group]  #
```

与 List 消息队列的区别：

- 消息持久化，消费后不丢失
- 支持多消费者组独立消费同一条流
- 有 ACK 机制，消费失败可重试
- 有消息 ID，可以回溯

Pub/Sub

发布订阅，消息即发即弃，不持久化，离线的订阅者收不到。

```
SUBSCRIBE channel [channel ...] #
UNSUBSCRIBE [channel ...]       #
PUBLISH channel message         #
PSUBSCRIBE pattern [pattern ...]  #  news.*
```

需要可靠投递就用 Stream，Pub/Sub 适合实时通知这类丢了也无所谓的场景。

事务与脚本

```
#
MULTI                           #
......
EXEC                            #
DISCARD                         #
WATCH key [key ...]             # EXEC

# Lua
EVAL "lua" numkeys key [key ...] arg [arg ...]
EVALSHA sha1 numkeys ...        #  SHA1
EVALRO / EVALSHA_RO             # 7.0+

# Redis Functions7.0+ EVAL
FUNCTION LOAD [REPLACE] "library"
FCALL function numkeys key [key ...] arg [arg ...]
FCALL_RO ...                    #
FUNCTION LIST / DELETE / DUMP / RESTORE
```

EVAL 里的 Lua 脚本在 Redis
单线程上原子执行，非常适合需要多步操作保持原子性的场景（比如「读→判断→写」）。7.0 的 Functions 把脚本当一等公民管理，比裸 EVAL 方便持久化和分发。

第一部分：数据结构与对象

简单动态字符串（SDS）

Redis 不用 C 原生字符串，而是自己造了 SDS（Simple Dynamic String）。数据库里所有字符串值的底层都是 SDS，AOF 缓冲区和客户端输入缓冲区也用它。

结构定义

```
struct sdshdr {
    int len;    //
    int free;   //
    char buf[]; //  '\0'
};
```

末尾的 `'\0'` 不计入 `len`，由 SDS 函数自动维护。好处是可以直接复用 C 标准库的部分函数（如 `printf("%s", s->buf)`）。

SDS vs C 字符串

特性	C 字符串	SDS
获取长度	O(N) 遍历	O(1) 读 `len`
缓冲区溢出	拼接前忘分配就溢出	API 自动检查并扩容
内存重分配	改 N 次必分配 N 次	改 N 次最多分配 N 次
数据类型	只能存文本	二进制安全，图片音频都行
兼容 C 函数	全部	部分（因为保留了 `'\0'`）

空间预分配

扩容时不只分配刚好够的空间，还会多分配一块：

- `len < 1MB`：额外分配与 `len` 等量的 `free` 空间（`buf` 实际大小 = `len + len + 1`）
- `len >= 1MB`：额外分配固定 1MB

效果：连续增长 N 次字符串，内存重分配次数从必定 N 次降到最多 N 次。

惰性空间释放

缩短字符串时不立即回收多余内存，而是记到 `free` 里留着以后用。需要时可以手动调 API 真正释放。

二进制安全

C 字符串遇到 `'\0'` 就认为结束了，SDS 用 `len` 判断长度，中间可以有任意字节。所以 SDS 能存任何二进制数据。

字典

Redis 用字典做数据库的底层实现，也用它做哈希键的底层实现。C 语言没有内置字典，Redis 自己写了一套。

数据结构

三层嵌套：

```
//
typedef struct dictEntry {
    void *key;
    union { void *val; uint64_t u64; int64_t s64; } v;
    struct dictEntry *next;  //
} dictEntry;

//
typedef struct dictht {
    dictEntry **table;      //
    unsigned long size;     //
    unsigned long sizemask; // size - 1
    unsigned long used;     //
} dictht;

//
typedef struct dict {
    dictType *type;    //
    void *privdata;
    dictht ht[2];      // ht[1]  rehash
    int rehashidx;      // rehash -1  rehash
} dict;
```

哈希算法

索引计算：`index = hash(key) & ht[x].sizemask`

Redis 对数据库和哈希键使用 MurmurHash2 算法，输入有规律也能给出好的随机分布。

键冲突处理

链地址法：同一个桶里的节点用 `next` 指针串成单向链表。新节点插到链表头部（O(1)）。

rehash

负载因子 = `ht[0].used / ht[0].size`

触发条件：

- 没在执行 `BGSAVE`/`BGREWRITEAOF` 时，负载因子 >= 1 → 扩容

- 正在执行 `BGSAVE`/`BGREWRITEAOF` 时，负载因子 >= 5 → 扩容（阈值提高是因为 fork 子进程期间要避免写时复制开销）
- 负载因子 < 0.1 → 缩容

扩容大小：第一个 >= `ht[0].used * 2` 的 <font color="#D9534F">2<super>n</super></font>

缩容大小：第一个 >= `ht[0].used` 的 <font color="#D9534F">2<super>n</super></font>

渐进式 rehash

如果哈希表有几百万个键值对，一次性迁移会卡住服务。Redis 的方案是分步搬迁：

1. 给 `ht[1]` 分配空间，`rehashidx` 设为 0
2. 每次对字典执行增删查改，顺带把 `ht[0]` 在 `rehashidx` 索引上的所有键值对迁移到 `ht[1]`，然后 `rehashidx++`
3. 全部搬完后，释放 `ht[0]`，把 `ht[1]` 变成 `ht[0]`，新建空的 `ht[1]`，`rehashidx` 设回 -1

rehash 期间的操作规则：

- 查找/删除/更新：先查 `ht[0]`，没找到再查 `ht[1]`
- 新增：只往 `ht[1]` 写，保证 `ht[0]` 只减不增

跳跃表

Redis 用跳跃表做有序集合（Sorted Set）的底层实现之一，也在集群节点内部用到。别的地方不用。

跳跃表查找平均 O(log N)，最坏 O(N)，实现比平衡树简单。

数据结构

```
typedef struct zskiplistNode {
    struct zskiplistNode *backward; //
    double score;                   //
    robj *obj;                      // SDS
    struct zskiplistLevel {
        struct zskiplistNode *forward; //
        unsigned int span;             //
    } level[]; //  1~32
} zskiplistNode;

typedef struct zskiplist {
    struct zskiplistNode *header, *tail; //
    unsigned long length;                //
    int level;                           //
} zskiplist;
```

核心概念

层（level）： 每个节点创建时随机生成 1~32
层（幂次定律，层越高概率越小）。层越多，跳过的节点越多，查找越快。

前进指针： 从表头向表尾方向遍历。高层指针跳得远，低层指针跳得近。

跨度（span）： 不是用来遍历的，是用来算排位（rank）的。查找过程中把沿途所有层的跨度加起来，就是目标节点的排位。

后退指针： 每个节点只有一个，只能回退到前一个节点。用于从表尾向表头遍历。

排序规则： 先按分值从小到大排，分值相同按成员对象的字典序排。成员对象必须唯一，分值可以重复。

压缩列表

压缩列表（ziplist）是 Redis
为省内存造的，一段连续内存块，用作列表键和哈希键在数据量小时的底层实现。

内存布局

```
| zlbytes | zltail | zllen | entry1 | entry2 | ... | zlend |
```

字段	大小	作用
`zlbytes`	4 字节	整个压缩列表占用的内存字节数
`zltail`	4 字节	尾节点距起始地址的偏移量（直接定位尾节点，O(1)）
`zllen`	2 字节	节点数量（>= 65535 时需要遍历才能得到真实数量）
`zlend`	1 字节	固定 `0xFF`，标记末尾

节点结构

每个 entry 由三部分组成：

```
| previous_entry_length | encoding | content |
```

`previous_entry_length`： 前一个节点的长度。前节点 < 254 字节时用 1 字节存，>= 254 字节时用 5 字节存（首字节 `0xFE` + 4 字节长度）。靠这个字段可以从尾到头反向遍历。

`encoding`： 标识 content 的数据类型和长度。高位 `00`/`01`/`10` 表示字节数组，`11` 开头表示整数。

`content`： 实际数据，字节数组或整数。

连锁更新

假设有一串连续节点，每个长度都在 250~253 字节。这时在表头插入一个 >= 254 字节的新节点：

1. 第一个节点的 `previous_entry_length` 要从 1 字节扩到 5 字节
2. 扩完之后这个节点本身长度突破 254 字节
3. 下一个节点的 `previous_entry_length` 也得扩……依此类推

这就是连锁更新，最坏 $O(N^2)$。删除节点也可能触发。

实际上这种场景很少见（需要一堆 250~253 字节的连续节点），所以 `ziplistPush`
等操作的平均复杂度仍然是 O(N)。

对象系统

Redis 不直接用底层数据结构存数据，而是封装了一层对象系统。每个键值对的键和值都是一个 `redisObject`：

```
typedef struct redisObject {
    unsigned type:4;     //
    unsigned encoding:4; //
    void *ptr;           //
    int refcount;        //
    unsigned lru:22;     //
} robj;
```

五种类型和编码

键固定是字符串对象，值可以是以下五种类型。每种类型根据数据量自动选择不同的底层编码：

类型	编码	底层实现	使用条件
字符串	`int`	long 整数	值是整数且能用 long 表示
字符串	`embstr`	一次分配的 SDS	字符串 <= 39 字节
字符串	`raw`	两次分配的 SDS	字符串 > 39 字节
列表	`ziplist`	压缩列表	元素数 < 512 且每个元素 < 64 字节
列表	`linkedlist`	双端链表	超出上述限制
哈希	`ziplist`	压缩列表	键值对数 < 512 且键值都 < 64 字节
哈希	`hashtable`	字典	超出上述限制
集合	`intset`	整数集合	全是整数且元素数 <= 512
集合	`hashtable`	字典（值全为 NULL）	有非整数元素或超 512 个
有序集合	`ziplist`	压缩列表	元素数 < 128 且成员 < 64 字节
有序集合	`skiplist`	跳表 + 字典	超出上述限制

数据量小时用紧凑结构省内存，数据量大时自动转换为高效结构。转换单向不可逆。

字符串对象的三种编码

- `int`： 直接把整数值存在 `ptr` 里（把指针当整数用）
- `embstr`： `redisObject` 和 SDS 分配在一块连续内存里，只需一次 `malloc`/`free`，缓存友好。`embstr` 是只读的，一旦修改就会转成 `raw`
- `raw`： `redisObject` 和 SDS 分开分配，两次 `malloc`

浮点数以字符串形式存储，运算时临时转回浮点数。

有序集合为什么同时用跳表和字典

`zset` 结构里同时维护了跳表和字典：

```
typedef struct zset {
    zskiplist *zsl; // ZRANGEZRANK
    dict *dict;     // O(1) ZSCORE
} zset;
```

单用字典做不了范围查询（无序），单用跳表查单个成员要 O(log N)。两者共享成员和分值的指针，不会浪费额外内存。

内存回收

引用计数：创建时 `refcount=1`，被引用 +1，不再引用 -1，降到 0 就释放。

对象共享

Redis 启动时预创建 0~9999 的字符串对象。多个键的值如果是这个范围内的整数，直接共享同一个对象（`refcount` 对应增加）。

不共享字符串对象，因为验证两个字符串是否相同是 O(N)，不划算。

LRU 空转时长

`lru` 字段记录最后一次访问时间。`OBJECT IDLETIME` 命令可以查看空转时长。

当 `maxmemory` 开启且淘汰策略为 `volatile-lru` 或 `allkeys-lru` 时，空转时间长的键会被优先淘汰。

第二部分：单机数据库

数据库

数据库结构

Redis 服务器默认创建 16 个数据库（`dbnum` 配置项），用 `SELECT` 命令切换。每个 `redisDb` 结构有两个关键字典：

- 键空间（`dict *dict`）：存所有键值对，键是字符串对象，值是任意类型对象
- 过期字典（`dict *expires`）：键指向键空间中的键对象，值是毫秒级 UNIX 时间戳

`EXPIRE`/`PEXPIRE`/`EXPIREAT`/`PEXPIREAT` 四个命令底层都转换为 `PEXPIREAT` 执行。

过期键删除策略

三种方案：

策略	优点	缺点
定时删除	内存友好，到期即删	CPU 不友好，创建大量定时器
惰性删除	CPU 友好，只在访问时检查	内存可能泄漏，大量过期键堆积
定期删除	折中	频率和时长需要调参

Redis 采用惰性删除 + 定期删除组合：

- 惰性删除： 所有读写命令执行前调用 `expireIfNeeded()`，如果键过期就删除
- 定期删除： `serverCron` 周期性调用 `activeExpireCycle()`，每次随机检查若干数据库，每个库随机取 20 个键检查

过期键与持久化、复制

- RDB 生成：跳过过期键
- AOF 重写：跳过过期键
- 主从复制：主服务器删除过期键后向从服务器发 `DEL` 命令；从服务器不会自主删除过期键，必须等主服务器通知

RDB 持久化

RDB 把某个时间点的数据库快照保存为二进制文件。

SAVE vs BGSAVE

	SAVE	BGSAVE
执行方式	主进程直接执行	fork 子进程执行
是否阻塞	阻塞，拒绝所有命令	不阻塞，父进程继续处理请求
互斥	—	执行期间拒绝再次 SAVE/BGSAVE；BGREWRITEAOF 延迟到 BGSAVE 结束后

载入 RDB 文件是在服务器启动时自动进行的。如果开启了 AOF，优先用 AOF 还原；AOF 关闭才用 RDB。

自动保存

默认配置（任一条件满足就触发 `BGSAVE`）：

```
save 900 1      # 900  1
save 300 10     # 300  10
save 60 10000   # 60  10000
```

实现：`dirty` 计数器记录上次保存后的修改次数，`lastsave` 记录上次保存时间。`serverCron` 每 100ms 检查一次条件。

RDB 文件结构

```
[REDIS][db_version][databases][EOF][check_sum]
```

- `REDIS`：5 字节魔数
- `db_version`：4 字节版本号
- `databases`：每个非空数据库的数据
  - `SELECTDB | db_number | key_value_pairs`
  - 键值对可带过期时间：`EXPIRETIME_MS | ms_timestamp | TYPE | key | value`
- `EOF`：1 字节结束标记
- `check_sum`：8 字节校验和

AOF 持久化

AOF 通过保存写命令来记录数据库状态，恢复时逐条重放。

工作流程

三步：追加、写入、同步。

1. 追加：每条写命令执行后，以 RESP 协议格式追加到 `aof_buf` 缓冲区
2. 写入：事件循环结束前，调用 `write` 将 `aof_buf` 写到内核缓冲区
3. 同步：根据 `appendfsync` 决定何时 `fsync` 到磁盘

appendfsync 策略

选项	行为	效率	安全性
`always`	每次写入后立即 fsync	最慢	最多丢一条命令
`everysec`（默认）	每秒 fsync 一次（后台线程）	高	最多丢一秒数据
`no`	不主动 fsync，交给 OS	最快	取决于 OS，通常 30s

AOF 重写

AOF 文件会不断膨胀。重写通过读取当前数据库状态，用最少命令重新生成 AOF 文件，不会读取旧 AOF。

比如一个列表经过 6 次 RPUSH，重写后变成一条 RPUSH
包含所有元素。单条命令元素过多时，会拆成多条（阈值 64 个）。

BGREWRITEAOF

fork 子进程执行重写。重写期间父进程照常处理请求，新的写命令同时追加到：

- AOF 缓冲区：维护现有 AOF 文件
- AOF 重写缓冲区：记录重写开始后的增量

子进程重写完成后，父进程把重写缓冲区的内容追加到新 AOF 文件，再原子替换旧文件。

事件

Redis 是事件驱动的，靠文件事件和时间事件运转。

文件事件

Redis 用 I/O 多路复用监听多个套接字，采用 Reactor 模式：

```
  → I/O  →  →  →
```

多路复用的底层实现可以是 `epoll`、`kqueue`、`evport`、`select`，编译时自动选最高效的。多个套接字就绪时逐个处理，同一套接字优先处理读事件再处理写事件。

常用事件处理器

- 连接应答处理器：监听套接字的 `AE_READABLE` 事件，接受新连接
- 命令请求处理器：客户端套接字的 `AE_READABLE` 事件，读取并执行命令
- 命令回复处理器：客户端套接字的 `AE_WRITABLE` 事件，返回结果给客户端

一次完整的客户端交互：

1. 客户端连接 → 触发连接应答处理器 → 创建客户端套接字，绑定命令请求处理器
2. 客户端发命令 → 触发命令请求处理器 → 执行命令，绑定命令回复处理器
3. 客户端可读 → 触发命令回复处理器 → 写回结果，解除绑定

时间事件

时间事件存放在无序链表中（数量少，遍历不影响性能），分两类：

- 定时事件：指定时间执行一次
- 周期事件：每隔一段时间执行（处理器返回下次间隔毫秒数）

`serverCron` 是 Redis 最核心的周期时间事件（默认 100ms 一次），负责：

- 更新统计信息（内存占用、连接数等）
- 清理过期键
- 检查自动保存条件
- 进行 AOF/RDB 相关后台操作
- 集群模式下的定期同步

事件调度

事件循环中，先计算最近的时间事件截止时间作为阻塞上限，然后阻塞等待文件事件。文件事件到达就处理，之后检查并执行到期的时间事件。文件事件和时间事件都是同步、有序、原子执行，不会互相抢占。

事务

Redis 事务把多条命令打包，按顺序一口气执行完，中间不会被别的客户端插队。

基本流程

```
MULTI       →
1       →
2       →
...
EXEC        →
```

客户端有 `REDIS_MULTI` 标志位表示当前处于事务状态。除了 `EXEC`、`DISCARD`、`WATCH`、`MULTI` 四条命令外，其他命令全部放进事务队列。

WATCH — 乐观锁

`WATCH` 在 `MULTI` 之前调用，监视任意数量的键。如果被监视的键在 `EXEC` 之前被别的客户端修改了，事务拒绝执行，`EXEC` 返回空。

实现：每个数据库维护 `watched_keys` 字典，键→监视该键的客户端链表。任何对数据库键的修改操作都会遍历 `watched_keys`，把相关客户端的 `REDIS_DIRTY_CAS` 标志打开。`EXEC` 时检查此标志，若已打开则拒绝执行。

ACID 分析

特性	支持情况
原子性	命令要么全入队要么不入队。但单条命令执行出错不会回滚，已成功的命令不撤销
一致性	入队错误拒绝执行；运行错误不影响正确命令的结果；宕机后靠 RDB/AOF 恢复一致
隔离性	单线程串行执行，天然隔离
持久性	取决于持久化配置。无持久化或 RDB 模式下不保证；AOF always 模式下有持久性

Redis 事务没有回滚。官方的态度是：事务里出错多半是你代码写错了，不该在运行时才发现，所以干脆不做回滚，保持实现简单。

第三部分：多机数据库

复制

执行 `SLAVEOF` 就能让一台 Redis 变成另一台的从服务器，同步数据。

旧版复制（2.8 之前）

只有 `SYNC` 命令，流程：

1. 从服务器发 `SYNC`
2. 主服务器执行 `BGSAVE` 生成 RDB 文件，同时用缓冲区记录此后的写命令
3. 主服务器把 RDB 文件发给从服务器，从服务器载入
4. 主服务器把缓冲区中的写命令发给从服务器执行

之后进入命令传播阶段：主服务器每执行写命令就发一份给从服务器。

缺陷：从服务器断线重连后必须重新完整 SYNC，即使只断了几秒。BGSAVE 非常耗资源（CPU、内存、磁盘 I/O、带宽），短暂断线后全量同步浪费严重。

新版复制（2.8+，PSYNC）

`PSYNC` 支持两种模式：

- 完整重同步：和旧版 SYNC 一样，用于首次复制
- 部分重同步：断线重连后只补发缺失部分

部分重同步的三个组件

复制偏移量（`offset`）：主从各自维护。主服务器每传播 N 字节，偏移量加 N；从服务器每收到 N 字节，偏移量加 N。比较两者偏移量就知道有没有断开过。

复制积压缓冲区：主服务器维护的固定大小（默认 1MB）FIFO 队列，保存最近传播的命令及其偏移量。从服务器重连时上报自己的 offset，如果 offset+1 的数据还在缓冲区内，就执行部分重同步；否则退回完整重同步。

服务器运行 ID（`runID`）：每个 Redis 实例启动时生成 40 位随机 ID。从服务器记住上次对接的主服务器 runID。重连时比对，runID 一致才有可能部分重同步；不一致（换了主服务器）直接完整重同步。

PSYNC 决策逻辑

```
→ PSYNC ? -1              //
→ PSYNC &lt;runID&gt; &lt;offset&gt;  //


 +FULLRESYNC &lt;runID&gt; &lt;offset&gt;     //
 +CONTINUE                        //
```

命令传播阶段

从服务器默认每秒发 `REPLCONF ACK <offset>` 给主服务器，作用：

- 检测网络连接状态
- 辅助 `min-slaves` 配置（保证最少 N 个从服务器延迟不超过 M 秒才接受写入）
- 检测命令丢失（主服务器对比 offset，发现差异就补发）

Sentinel

Sentinel（哨兵）监控 Redis 主从服务器，主服务器挂了就自动做故障转移。

启动与初始化

Sentinel 本质上是运行在特殊模式下的 Redis 服务器，不使用数据库功能，不载入 RDB/AOF。启动后根据配置文件中的主服务器地址建立连接，自动发现该主服务器的所有从服务器和其他 Sentinel。

每个 Sentinel 对每个被监控的主/从服务器建立两条连接：

- 命令连接：发送命令并接收回复
- 订阅连接：订阅 `__sentinel__:hello` 频道，用于 Sentinel 之间互相发现

监控

- 每 10 秒向主/从服务器发 `INFO`，获取当前信息（从服务器列表、角色、偏移量等）
- 每 2 秒通过 `__sentinel__:hello` 频道发布自己的信息和主服务器信息
- 每 1 秒向所有主/从服务器和其他 Sentinel 发 `PING`，检测存活

下线判断

主观下线（SDOWN）：一个 Sentinel 在 `down-after-milliseconds` 内没收到有效回复，判定目标主观下线。

客观下线（ODOWN）：Sentinel 向其他 Sentinel 询问（`SENTINEL is-master-down-by-addr`），如果足够多数量的 Sentinel 都同意下线（`quorum` 配置），判定客观下线。客观下线判断只针对主服务器。

选举 Leader

主服务器被判定客观下线后，Sentinel 之间需要选出一个 Leader 来执行故障转移。算法和 Raft 的领头选举类似：

- 每个 Sentinel 在每个配置纪元中有一票，先到先得
- 收到投票请求后，把票投给第一个请求的 Sentinel
- 获得半数以上票的 Sentinel 成为 Leader
- 如果这一轮没选出来，等一段时间后重新开始

故障转移

Leader Sentinel 执行三步：

1. 选新主服务器：从下线主服务器的从服务器中挑选——排除断线的、最近没有回复 INFO 的，然后按优先级→复制偏移量→runID 排序，选最合适的
2. 让新主服务器转换角色：向选中的从服务器发 `SLAVEOF no one`，然后持续发 `INFO` 确认它已经变成主服务器
3. 让其他从服务器复制新主：向剩余从服务器发 `SLAVEOF < IP> <>`
4. 将旧主设为从服务器：在旧主服务器的实例结构中设置目标，等它重新上线时向它发 `SLAVEOF`

集群

Redis Cluster 把数据分片存到多个节点上，节点之间互为备份。

节点

每个节点就是一个运行在集群模式下的 Redis 服务器。通过 `CLUSTER MEET <ip> <port>` 让节点握手加入集群。节点之间用 Gossip 协议通信，信息在集群中扩散传播。

槽指派

集群一共有 16384 个槽（slot），所有槽全部分配完毕后集群才能上线。

每个节点用一个 16384 位的位数组 `slots` 记录自己负责哪些槽——第 i 位为 1 表示负责槽 i。同时 `clusterState.slots[i]` 数组直接指向负责该槽的节点，查找 O(1)。

键到槽的映射：

```
slot = CRC16(key) &amp; 16383
```

MOVED 与 ASK

MOVED：客户端请求的键不在当前节点。节点回复 `MOVED <slot> <ip>:<port>`，客户端转向目标节点重新请求，并更新本地槽映射缓存。

ASK：槽正在迁移中，键已经转移到目标节点。回复 `ASK <slot> <ip>:<port>`，客户端先向目标节点发 `ASKING`，再发命令。ASK 是一次性的，不更新本地缓存。

重新分片

在线操作，不需要停机。由 `redis-trib` 工具驱动：

1. 通知目标节点准备导入槽（`CLUSTER SETSLOT <slot> IMPORTING <source_id>`）
2. 通知源节点准备迁出槽（`CLUSTER SETSLOT <slot> MIGRATING <target_id>`）
3. 源节点逐个获取键（`CLUSTER GETKEYSINSLOT`），然后 `MIGRATE` 到目标节点
4. 全部迁移完毕后 `CLUSTER SETSLOT <slot> NODE <target_id>` 通知所有节点

故障检测

集群节点定期互相发 `PING`：

- PFAIL（疑似下线）：节点 A 在超时时间内没收到节点 B 的回复，A 把 B 标记为 PFAIL
- FAIL（已下线）：半数以上负责槽的主节点都把 B 标记为 PFAIL，那么 B 被标记为 FAIL，并广播到整个集群

故障转移

下线主节点的从节点发起选举：

1. 从节点广播 `CLUSTERMSG_TYPE_FAILOVER_AUTH_REQUEST`
2. 只有负责槽的主节点有投票权，每个配置纪元一票
3. 获得半数以上票的从节点成为新主
4. 新主接管原主的所有槽并广播 `PONG`

从节点选举优先级：复制偏移量最大的（数据最新的）从节点更容易先发起选举。

Gossip 协议

节点间通过 Gossip 消息交换状态：

消息类型	用途
MEET	通知接收方加入集群
PING	检测在线状态，随机选若干节点
PONG	回复 MEET/PING，或广播自身信息
FAIL	广播某节点已下线
PUBLISH	向集群所有节点广播消息